Nome: Bianca Banaletti e Sabrina Figueiredo
Disciplina: Sistemas Operacionais II

Resumo da aula do dia 05/08

1. Processos
2. Programação concorrente
3. Criação e gerência de processos
4. Threads

Processos: São o conjunto necessário de informações para que o sistema operacional implemente a ocorrência de programas. O gerenciamento de é feito exclusivamente pelo sistema operacional, que deve controlar a execução dos diversos programas e o uso concorrente do processador e demais recursos.
A gerência de processo é uma das principais funções do sistema operacional, possibilitando aos programas alocar recursos, compartilhar dados, trocar informações, entre outros. Nos sistemas multiprogramáveis os processos compartilham o uso do processador, memória principal e dispositivos de E/S.

Programação concorrente: Para que a concorrência entre processos ocorra sem problemas, é necessário que as informações do programa que foi interrompido sejam armazenadas para que quando o programa voltar a ser executado, não falte nenhuma informação necessária para continuar o processamento.

Criação e gerência de processos: É o módulo principal de funcionamento do sistema operacional, responsável por transformar programas armazenados no disco em tarefas ativas na memória, controlando sua execução, seus estados e a divisão dos recursos da máquina. Atua como um maestro, garantindo que múltiplos programas em execução dividam os recursos do computador (como CPU e memória) de forma justa, segura e sem conflitos.

Processos e Threads: Um processo é um programa em execução com seu próprio espaço de memória isolado, enquanto uma thread é uma unidade menor de execução que compartilha a mesma memória do processo principal.

- A correlação entre processo e thread em uma relação de 1 para muitos: um processo pode conter múltiplas threads, mas uma thread não pode existir fora de um processo. O processo atua como o contentor de recursos e o ambiente de execução, enquanto as threads são os agentes que executam o código dentro desse ambiente.

Threads: A partir do conceito de múltiplos threads, é possível implementar aplicações concorrentes de forma eficiente, pois um processo pode ter partes diferentes do seu código sendo executadas concorrentemente, com um menor overhead do que utilizando múltiplos processos.

Threads de um mesmo processo compartilham o mesmo espaço de endereçamento, então a comunicação entre threads não envolve mecanismos lentos entre processos, aumentando o desempenho da aplicação.

Programação concorrente: É um modelo de desenvolvimento onde múltiplas tarefas são iniciadas, executadas e concluídas em períodos de tempo que se sobrepõem, utilizando threads como a menor unidade de execução independente dentro de um mesmo processo. Enquanto um processo possui o seu próprio espaço de memória isolado, as threads de um mesmo processo partilham a mesma memória, ficheiros e variáveis.

Desafios:

- Condição de Corrida (Race Condition): Quando duas threads tentam alterar a mesma variável em simultâneo, gerando dados inconsistentes.
- Deadlock (Impasses): Situação onde uma Thread A espera pela Thread B, e a Thread B espera pela Thread A, bloqueando o sistema permanentemente.
- Inanição (Starvation): Uma thread nunca consegue tempo de CPU porque outras têm prioridade máxima constante.

Criação e gerência de processos: Varia conforme o sistema operativo. O sistema operativo gere processos como blocos isolados e as threads como fluxos de execução dentro deles. Atualmente, a gerência eficiente evita a criação manual constante de threads, utilizando pools de threads para reaproveitar recursos.

Ciclo:

- Criação do Processo: O SO aloca memória (heap, stack) e recursos exclusivos.
- Criação da Thread: Criada dentro do processo, partilha o heap, mas ganha a sua própria stack.
- Estado Pronto (Ready): A thread aguarda que o escalonador do SO lhe atribua tempo de CPU.
- Estado Executando (Running): O código da thread está a ser processado pelo núcleo do CPU.
- Estado Bloqueado (Blocked/Waiting): A thread espera por E/S (I/O) ou por um mecanismo de sincronização.
- Estado Terminado (Terminated): A execução acaba e os recursos locais da thread são limpos.